

自作DirectX12農場ゲーム / Editor Debug Environment

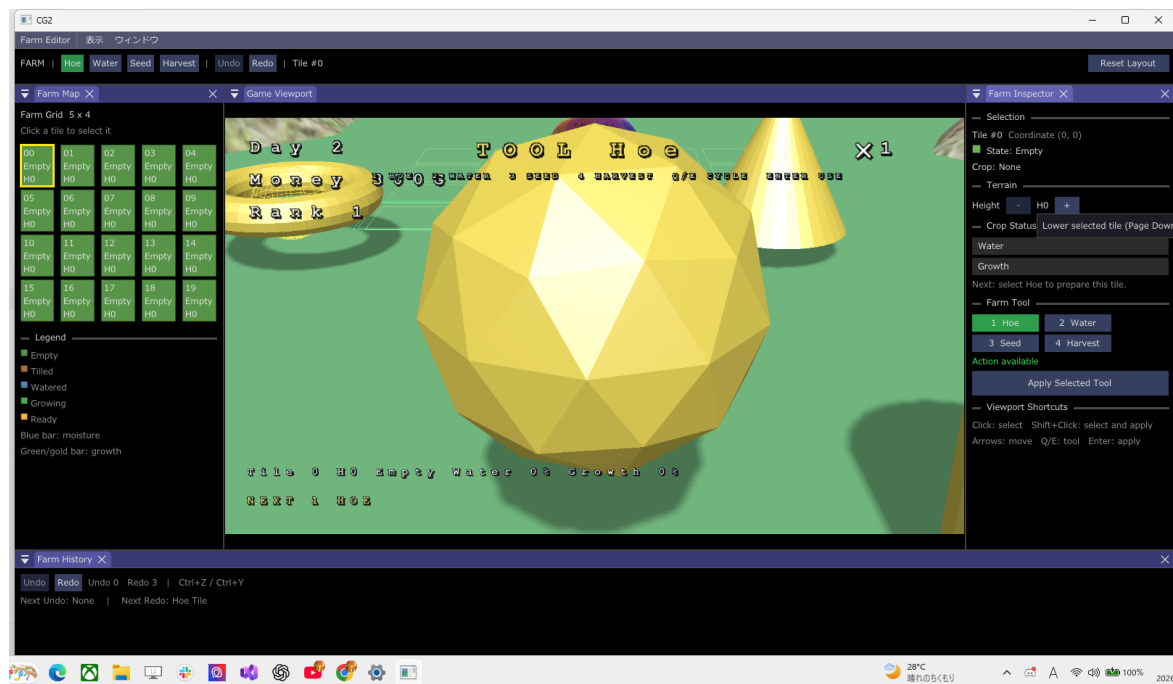
農場ゲームのコアループと、自作エンジン上のEditor Debug環境をまとめたポートフォリオ資料

ヨシノ ゲント / LE3C_26_ヨシノ_гент / 2026年8月



作品概要	開発内容	技術アピール
DirectX12 / C++で制作している農場ゲーム。畑マスを選び、耕す・水やり・種まき・収穫の流れを確認できるプレイアブルな検証版です。	FarmGrid、ToolActionSystem、HUD、Debug Editorを分離し、ゲーム状態と表示/UIを混ぜない設計を重視しました。	DX12描画基盤、Sprite/BitmapFont UI、ImGui Debug Editor、JSON保存、Animation/Skinning検証を組み合わせています。

1. ゲームとして確認できる要素



農場ゲームの基本操作

- 畑マスを選択し、選択中Tileの状態をHUDとInspectorへ反映
- Hoe / Water / Seed / Harvest
の操作をToolActionSystem経由で適用
- Day、Tool、倍率、選択Tile情報などをHUDで表示
- Debug UIから内部状態を確認しながら動作検証可能

操作	結果
畑マス選択	選択中TileのIndex / 高さ / 状態 / 水分 / 作物を表示
Hoe	EmptyのTileをTilledに変更
Water	Tilled / PlantedのTileの水分を100%に変更
Seed	TilledのTileにTestCropを植える
Harvest	成長済み作物の収穫状態を確認

2. 設計上のこだわり

状態管理

FarmGridがTile配列と選択状態を保持。描画やHUD表示は持たせず、論理データだけを担当。

Debug Editor

ImGuiでFarm状態、Object選択、Animation確認を整理。開発中の検証速度を上げる。

ツール効果

FarmToolActionSystemにHoe / Water / Seed / Harvestの変更処理を集約。Sceneに農場ロジックを直書きしない。

データ保存

JsonFileを使い、FarmDocumentの保存/読み込みを分離。ゲーム本編とEditor設定を混同しない。

表示

FarmHUDはViewDataを受け取って描画するだけ。ゲーム状態を直接変更しない。

低レイヤ

DX12のDescriptor、Resource、Shader、Sprite、3D描画基盤を自作エンジン側で管理。

今後の改善点

- 作物種類、売却、所持金、ゲーム目標を追加してコアループを完成させる
- Editor UIをGodot風のSceneTree / Inspector / Asset管理へ段階的に整理する
- DX12 Resource管理、Descriptor寿命、RenderGraph化など低レイヤの保守性を高める

3. 提出物と確認ポイント

項目	内容
実行ファイル	Release版exeとResources、DLL、CSOを同梱
ソースコード	Visual Studio Solution、C++ソース、Shaders、Resources、Settingsを同梱
作品紹介動画	農場ゲームの操作とEditor確認をテロップ付きで説明
プログラム説明資料	クラス責務、処理順、ゲーム状態変更の流れを記載

